In [1]:
```
import nltk
```

In [2]:
```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\System05\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt.zip.
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\System05\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\System05\AppData\Roaming\nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     C:\Users\System05\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger.zip.
```

Out[2]: True

In [3]:
```
text="Tokenization is the first step in text analytics.The process of breaking down
```

In [4]:
```
from nltk.tokenize import sent_tokenize
```

In [6]:
```
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\System05\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping tokenizers\punkt_tab.zip.
```

Out[6]: True

In [7]:
```
sentences=sent_tokenize(text)
print("Sentence Tokenization:",sentences)
```

```
Sentence Tokenization: ['Tokenization is the first step in text analytics.The proces
s of breaking down a text paragraph into smaller chunks such as words or sentences i
s called Tokenization.']
```

In [8]:
```
from nltk.tokenize import word_tokenize
```

In [10]:
```
words=word_tokenize(text)
print("Word Tokenization:",words)
```

```
Word Tokenization: ['Tokenization', 'is', 'the', 'first', 'step', 'in', 'text', 'ana
lytics.The', 'process', 'of', 'breaking', 'down', 'a', 'text', 'paragraph', 'into',
'smaller', 'chunks', 'such', 'as', 'words', 'or', 'sentences', 'is', 'called', 'Toke
nization', '.']
```

In [11]:
```
import re
```

In [12]:
```
from nltk.corpus import stopwords
```

In [13]:
```
stop_words = set(stopwords.words("english"))
```

In [14]:
```
text2 = "How to remove stop words with NLTK library in Python?"
text2 = re.sub('[^a-zA-Z]', ' ', text2)
```

In [15]:
```
tokens = word_tokenize(text2.lower())
```

In [16]:
```
filtered_text = []
for w in tokens:
    if w not in stop_words:
        filtered_text.append(w)
```

In [17]:
```
print("Tokens:", tokens)
print("Filtered (Stopwords Removed):", filtered_text)
```

```
Tokens: ['how', 'to', 'remove', 'stop', 'words', 'with', 'nltk', 'library', 'in', 'p
ython']
Filtered (Stopwords Removed): ['remove', 'stop', 'words', 'nltk', 'library', 'pytho
n']
```

In [18]:
```
from nltk.stem import PorterStemmer
```

In [19]:
```
ps = PorterStemmer()
words = ["wait","waiting","waited","waits"]
```

In [20]:
```
for w in words:
    print(w, "->", ps.stem(w))
```

```
wait -> wait
waiting -> wait
waited -> wait
waits -> wait
```

In [21]:
```
from nltk.stem import WordNetLemmatizer
```

In [22]:
```
lemmatizer = WordNetLemmatizer()
```

In [23]:
```
text3 = "studies studying cries cry"
tokens = word_tokenize(text3)
```

In [24]:
```
for w in tokens:
    print("Lemma for", w, "is", lemmatizer.lemmatize(w))
```

```
Lemma for studies is study
Lemma for studying is studying
Lemma for cries is cry
Lemma for cry is cry
```

In [25]:
```
from nltk.tokenize import word_tokenize
```

In [26]:
```
data = "The pink sweater fit her perfectly"
words = word_tokenize(data)
```

In [28]:
```
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\System05\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger_eng.zip.
```

Out[28]: True

In [29]:
```
for word in words:
    print(word, "->", nltk.pos_tag([word]))
```

```
The -> [('The', 'DT')]
pink -> [('pink', 'NN')]
sweater -> [('sweater', 'NN')]
fit -> [('fit', 'NN')]
her -> [('her', 'PRP$')]
perfectly -> [('perfectly', 'RB')]
```

In [30]:
```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
import math
```

In [31]:
```
documentA ="Jupiter is the largest planet"
documentB ="Mars is the fourth planet from the sun"
```

In [32]:
```
bagOfWordsA =documentA.split()
bagOfWordsB =documentB.split()
```

In [33]:
```
uniqueWords =set(bagOfWordsA).union(set(bagOfWordsB))
```

In [34]:
```
numOfWordsA = dict.fromkeys(uniqueWords, 0)
for word in bagOfWordsA:
    numOfWordsA[word] += 1
```

In [35]:
```
numOfWordsB = dict.fromkeys(uniqueWords, 0)
for word in bagOfWordsB:
    numOfWordsB[word] += 1
```

In [39]:
```
def computeTF(wordDict, bagOfWords):
    tfDict = {}
    bagCount = len(bagOfWords)
    for word, count in wordDict.items():
        tfDict[word] = count / float(bagCount)
    return tfDict
```

In [40]:
```
tfA = computeTF(numOfWordsA, bagOfWordsA)
tfB = computeTF(numOfWordsB, bagOfWordsB)
```

In [41]:
```
def computeIDF(documents):
    N = len(documents)
    idfDict = dict.fromkeys(documents[0].keys(), 0)
    for document in documents:
        for word, val in document.items():
            if val > 0:
```

```
                idfDict[word] += 1
    for word, val in idfDict.items():
        idfDict[word] = math.log(N / float(val))
    return idfDict
```

In [42]:
```
idfs = computeIDF([numOfWordsA, numOfWordsB])
```

In [43]:
```
def computeTFIDF(tfDict, idfs):
    tfidf = {}
    for word, val in tfDict.items():
        tfidf[word] = val * idfs[word]
    return tfidf
```

In [44]:
```
tfidfA = computeTFIDF(tfA, idfs)
tfidfB = computeTFIDF(tfB, idfs)
```

In [45]:
```
df = pd.DataFrame([tfidfA, tfidfB])
print(df)
```

```
   planet    fourth  the   largest   is      Mars   Jupiter       sun  \
0     0.0  0.000000  0.0  0.138629  0.0  0.000000  0.138629  0.000000
1     0.0  0.086643  0.0  0.000000  0.0  0.086643  0.000000  0.086643

       from
0  0.000000
1  0.086643
```